NEW HORIZON
COLLEGE OF ENGINEERING
Autonomous College Permanently Affiliated to VTU, Approved by AICTE & UGC
Accredited by NAAC with 'A' Grade, Accredited by NBA

A PROJECT REPORT

for

Mini Project – II (22CSE58)

on

Automated RCA For Network Failure

Submitted by

AALIYA KHAN

USN: 1NH23CS005, Sem-Sec: 5-A

AMOGH LOKESH

USN: 1NH23CS024, Sem-Sec: 5-A

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Academic Year: 2025-26

NEW HORIZON
COLLEGE OF ENGINEERING
Autonomous College Permanently Affiliated to VTU, Approved by AICTE & UGC
Accredited by NAAC with 'A' Grade, Accredited by NBA

CERTIFICATE

This is to certify that the Mini Project-II (22CSE58) work entitled "**Automated RCA For Network Failure"** has been carried out and submitted by **AALIYA KHAN (1NH23CS005) and Amogh Lokesh (1NH23CS024)** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering** during the **Odd Semester of the Academic Year 2025-26.**

Signature of Reviewer **Signature of CSE HOD**

SEMESTER END EXAMINATION

Name of the Examiner(s) *Signature with date*

1. _____ _____
2. _____ _____

ABSTRACT

This project presents a robust framework for Automated Root Cause Analysis (RCA) designed to mitigate the operational challenges posed by increasingly complex network infrastructures, specifically addressing the critical issue of "Alert Fatigue." Traditional network monitoring often results in a chaotic flood of cascading alerts where a single device failure triggers a multitude of symptomatic notifications, obscuring the actual origin of the problem. To resolve this, the proposed system implements a multi-layered architecture featuring a unified data ingestion pipeline that normalizes diverse network logs and metrics, coupled with a Topology & Context Engine that dynamically maps network dependencies. The core intelligence is driven by a Correlation & Analytics Engine that leverages both rule-based logic and machine learning models to suppress noise and identify the fundamental source of failure. By transforming high-volume raw data into consolidated, actionable insights, the system significantly reduces the Mean Time to Resolution (MTTR). Furthermore, the dependency-based approach ensures the solution is highly scalable, capable of adapting to expanding network topologies without a corresponding increase in manual workload, ultimately ensuring greater network resilience and operational efficiency.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be impossible without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

I am delighted to express my gratitude to **Dr. Mohan Manghnani**, Chairman, New Horizon Educational Institutions, for furnishing the essential infrastructure and fostering a positive environment.

I would like to take this chance to express my deep gratitude to **Dr. Manjunatha,** Principal, New Horizon College of Engineering, for consistently offering support and encouragement.

I wish to convey my gratitude to **Dr. Anandhi R J**, Professor and Dean-Academics at NHCE, for providing indispensable guidance and unwavering support.

I want to express my heartfelt gratitude to **Dr. B. Rajalakshmi**, Professor and Head of the Department, Computer Science and Engineering, for the steadfast support that has remarkably shaped my academic journey.

I would like to extend my thanks to **Mr. Bhaskar S V** , Sr.Prof., Department of Computer Science and Engineering, who served as the reviewer for my mini project.

Aaliya Khan

USN:1NH23C005

Amogh Lokesh

USN:1NH23C024

CONTENTS

ABSTRACT **I**

ACKNOWLEDGEMENT **II**

CONTENTS **III**

LIST OF FIGURES **VI**

LIST OF TABLES **VI**

1. INTRODUCTION

1.1. AUTOMATED RCA FOR NETWORK FAILURE 1

1.2. THE PROBLEM STATEMENT 1

1.3. TECHNICAL APPROACH 2

1.4. KEY OBJECTIVES 4

1.5. SIGNIFICANCE OF THE PROJECT 4

2. FUNDAMENTALS OF PYTHON

2.1. WHAT IS PYTHON? 6

2.2. THE PYTHON PHILOSOPHY (OVERVIEW) 9

2.3. CORE DATA STRUCTURES 9

2.4. CONTROL FLOW & LOGIC 10

2.5. FUNCTIONS & MODULES 11

2.6. OBJECT-ORIENTED PROGRAMMING (OOP) 12

2.7. THE PYTHON PHILOSOPHY (OVERVIEW) 12

2.8. CORE DATA STRUCTURES 13

2.9. CONTROL FLOW & LOGIC 13

2.10. FUNCTIONS & MODULES 14

2.11. OBJECT-ORIENTED PROGRAMMING (OOP) 15
2.12. WHY THIS MATTERS FOR YOUR WORK 15

3. FUNDAMENTALS OF DBMS

3.1. WHAT IS A DBMS? 16
3.2. THE 3-LEVEL ARCHITECTURE (DATA ABSTRACTION) 16
3.3. KEY CHARACTERISTICS: THE ACID PROPERTIES **16**
3.4. DATA MODELS: SQL VS. NOSQL 17
3.5. SQL COMMAND CATEGORIES 18
3.6. RELEVANCE TO NETWORK RCA

4. FUNDAMENTALS OF SQL

4.1. THE CORE SYNTAX GROUPS 19
4.2. LOGICAL ORDER OF EXECUTION 20
4.3. THE POWER OF JOINS (RELATIONAL ALGEBRA) 20
4.4. AGGREGATION AND GROUPING 21
4.5. SUBQUERIES AND CTES 22
4.6. RELEVANCE TO YOUR NETWORK RCA PROJECT 23

5. DESIGN

5.1. DATA INGESTION & STREAMING 28
5.2. TOPOLOGY & CONTEXT ENGINE 28
5.3. CORRELATION & ANALYTICS ENGINE 28
5.4. RULES & ML MODEL REPOSITORY 28
5.5. VISUALIZATION DASHBOARD 29
5.6. INTEGRATION & NOTIFICATION GATEWAY 29

6. IMPLEMENTATION

6.1. Introduction 31
6.2. Development Environment & Tools 31
6.3. Implementation Models 31

6.4. Key Algorithms and Pseudocode 33
6.5. Sample Code Snippets 34
6.6. Challenges Faces 34
6.7. Conclusions of Implementations 34

7. RESULTS

7.1. ELIMINATES "ALERT STORMS" (NOISE REDUCTION) 35
7.2. DRASTICALLY REDUCES MEAN TIME TO RESOLUTION (MTTR) 35
7.3. SOLVES THE "SYMPTOM VS. CAUSE" CONFUSION 36
7.4. AUTOMATED CONTEXT 36
7.5. SUMMARY 36

8. CONCLUSION

8.1. PROBLEM SOLVED: ALERT FATIGUE 37
8.2. EFFICIENCY GAINED: FROM CHAOS TO ACTION 38
8.3. SCALABILITY: THE DEPENDENCY-BASED APPROACH 38

REFERENCES 39

LIST OF FIGURES

Figure No	Figure Description	Page No
1.1	RCA Process	1
1.2	Network Topology	2
5.1	Root Cause Analysis symptoms suppressed	23

LIST OF TABLES

Table No	Table Description	Page No
2.1	Core Data Structures	10
2.2	Core Data Structures	13
3.1	SQL Command Categories	18
4.1	Aggregate Functions	25
4.2	SQL vs NoSQL (Quick Difference)	27
7.1	Summary	39

CHAPTER 1

INTRODUCTION

1.1 AUTOMATED RCA FOR NETWORK FAILURE

In the era of Cloud Computing, 5G, and microservices, modern network infrastructures have become massively complex distributed systems. When a network failure occurs, it triggers a "storm" of alerts across various monitoring systems.

This project focuses on **Automated Root Cause Analysis (RCA)**. The objective is to move beyond simple fault detection (knowing *something* is wrong) to intelligent diagnosis (knowing *why* it is wrong) without human intervention. By leveraging data-driven algorithms, this system aims to pinpoint the fundamental source of a failure amidst thousands of symptomatic alarms.[1]

1.2. The Problem Statement

Modern network operations teams are currently hindered by two critical bottlenecks: Alert Fatigue and a high Mean Time to Resolution (MTTR). Alert fatigue arises when a single infrastructure failure, such as a router outage, triggers a cascade of secondary alarms across servers, databases, and applications, burying the actual issue under hundreds of redundant notifications. This leads to a high MTTR, as engineers are forced into a time-consuming and error-prone process of manually sifting through logs to find the "needle in the haystack." Ultimately, the core challenge is the system's inability to distinguish between a symptom, such as a database connection timeout, and the root cause, such as a specific switch port failure. Without an automated way to separate these layers, technical teams remain overwhelmed by the noise of the cascade rather than focused on the point of failure.

1.3. Technical Approach

This project proposes an automated solution that treats the network as a graph G = (V, E), where V represents nodes (servers, switches, routers) and E represents the links between them. The system utilizes the following methodologies: Topology Mapping: Constructing a dependency graph to understand how components interact. If Node A relies on Node B, a failure in B naturally explains errors in A. Correlation Algorithms: Using statistical methods or machine learning to correlate timestamps of events. Causal Inference: Moving from correlation to causation using logic-based rules or probabilistic models (like Bayesian Networks) to identify the source. This infographic displays six common network topologies, which describe how nodes (devices) and links (connections) are arranged in a computer network. Understanding these layouts is crucial for network design, as each topology has distinct advantages and disadvantages regarding cost, performance, and reliability.[1] [2] Mesh Topology: The first diagram illustrates a mesh topology, where every node is connected to every other node in the network. This design offers high reliability and redundancy, as there are multiple paths for data to travel. However, the extensive cabling required makes it expensive and complex to install and maintain, especially in large networks. Star Topology: Next is the star topology, characterized by a central hub to which all other nodes are connected. This is one of the most common network setups because it is easy to install and manage; if one node fails, the rest of the network remains operational. A significant drawback is that if the central hub fails, the entire network goes down. Bus Topology: To the right is the bus topology, where all nodes are connected to a single main cable, known as the bus or backbone. This linear arrangement is simple and cost-effective for small networks. However, a break in the main cable can disrupt the entire network, and performance degrades as more devices are added.[2] Ring Topology: Below the star and bus topologies is the ring topology, where each device is connected to exactly two other devices, forming a circular pathway for signals. Data travels in one direction, passing through each device until it reaches its destination. While this design prevents data collisions, the failure of a single node can break the

ring and halt network traffic. Tree Topology: Next to the ring is the tree topology, which is a hybrid structure that combines characteristics of star and bus topologies. It features a root node, and all other nodes are linked to form a hierarchy, resembling a tree. This topology is scalable and allows for the easy expansion of the network, but a failure in the central hub of a segment can isolate that entire branch. Hybrid Topology: The final diagram at the bottom shows a hybrid topology, which integrates two or more different topologies to form a larger, more complex network. In this example, we see a combination of star, bus, ring, and mesh topologies. Hybrid networks are designed to leverage the strengths of various topologies and are often used in large organizations.[1] [2] [3]

1.4. Key Objectives

The proposed system architecture is built upon four foundational pillars designed to streamline network maintenance and incident response. The process begins with Ingestion, providing real-time collection of diverse telemetry including logs, performance metrics such as CPU and latency, and distributed traces. This raw data then undergoes Noise Reduction, where a sophisticated filtering layer discards false positives and suppresses symptomatic alerts that would otherwise distract the operator. Once the data is refined, the Localization engine analyzes the infrastructure's logical path to pinpoint the exact node or link responsible for the failure. Finally, the system provides Remediation Suggestions, an optional but high-value feature that leverages historical failure data to recommend specific fixes, such as service restarts or configuration rollbacks, further accelerating the recovery process.[2]

1.5. Significance of the Project

Automating RCA is critical for maintaining **Service Level Agreements (SLAs)**. By reducing the time taken to identify faults from hours to seconds, organizations can ensure high availability, improve user experience, and significantly reduce operational costs.[2] [3]

CHAPTER 2

FUNDAMENTALS OF PYTHON

To understand the theory behind using Python for Automated Root Cause Analysis (RCA) in networking, you must view Python not just as a scripting language, but as the "orchestrator" that connects disparate network data, applies logic, and identifies the single point of failure among thousands of alerts. Automated RCA generally follows a pipeline: Collection Here is the theoretical framework of how Python handles each stage.[1]

2.1. Data Collection: The Interface Layer

In theory, RCA cannot happen without real-time state data. Networks are heterogeneous (Cisco, Juniper, Arista, etc.). Python acts as a vendor-agnostic abstraction layer. Protocol Abstraction: You typically do not write raw TCP sockets. Python libraries handle the handshake, encryption (SSH), and session management.

SSH/CLI: Libraries like Netmiko or Paramiko allow Python to "login" to devices and run commands (show ip route, show interfaces) just like a human operator.

SNMP & Telemetry: Python uses libraries (like pysnmp) to poll OIDs or subscribe to streaming telemetry (gRPC) for live interface statistics. API Interaction: Modern SDN (Software Defined Networking) controllers expose REST APIs. Python’s requests library is the standard tool to fetch network state via HTTP GET/POST methods.[3]

2.2. Data Parsing: Structuring the Unstructured

A major theoretical challenge in network RCA is that devices often output "unstructured" text (CLI output) meant for humans, not machines. Python’s role here is Normalization. Regular Expressions (Regex): The theoretical foundation for parsing text. You define patterns (e.g., "Find a pattern that looks like an IP address followed

by the word 'Down'"). Python's re module is the engine for this. State Machines: For complex outputs, simple regex isn't enough. Tools like TextFSM use state machine theory to parse multi-line output into structured tables (dictionaries or JSON). Data Structures: Once parsed, Python converts the data into Lists (ordered sequences of events) or Dictionaries (key-value pairs representing device attributes), making it computationally accessible.[2]

2.3. Correlation: Graph Theory & Topology

At its core, Root Cause Analysis (RCA) leverages graph theory to transform a chaotic storm of network alerts into a single, actionable insight. By modeling the network as a mathematical graph G = (V, E), where vertices (V) represent hardware like routers or servers and edges (E) represent their physical or logical links, Python-based systems can map complex dependencies. Using libraries like NetworkX, the system performs topological traversals—such as Breadth-First Search (BFS)—to trace the path of a failure. This logic allows the system to distinguish between a root cause and its symptoms: if a core router fails and fifty downstream switches report as unreachable, the algorithm recognizes that because the switches rely on the router for connectivity, their "failure" is merely a logical consequence of the router's outage. [3]

2.4. Diagnosis: Deterministic vs. Probabilistic Logic

Once data is collected and mapped, Python applies algorithms to determine the cause.

A. Deterministic Logic (Rule-Based)

Deterministic logic, often referred to as rule-based logic, functions by applying explicit "If-Then-Else" structures derived from established domain knowledge. Grounded in the theory of Boolean Logic, this approach provides clear, predictable outcomes based on specific input conditions. For example, in a network diagnostic context, the system might evaluate interface errors alongside CPU usage to

determine a root cause. If interface errors exceed 1,000 while CPU usage remains low (below 10%), the rule logically identifies the issue as a physical fault, such as a "Bad Cable or SFP." However, if interface errors exceed 1,000 while CPU usage is high (above 90%), the logic distinguishes this as "Control Plane Congestion," allowing for precise automated troubleshooting.[2]

B. Probabilistic Logic (Machine Learning/AI)

Probabilistic logic is utilized when operational rules are too complex to be hard-coded, relying instead on **Machine Learning** and AI to interpret data. Grounded in the theory of statistical deviation and **Anomaly Detection**, this approach leverages libraries such as **Pandas** for time-series analysis and **Scikit-learn**. The method works by establishing a historical "baseline" of performance—for example, determining that normal traffic latency is 20ms. If a significant deviation occurs, such as latency spiking to 150ms, the system flags it as an anomaly. To objectively evaluate the criticality of the event, Python calculates the **Z-score** (the number of standard deviations from the mean), allowing the system to grade the severity of the anomaly based on statistical evidence rather than static thresholds..[1] [3]

CHAPTER 3

FUNDAMENTALS OF DBMS

In the theoretical framework of **Automated Root Cause Analysis (RCA)**, the relationship between processing and storage is defined by distinct roles: if Python serves as the "processor" or "logic engine," the **DBMS (Database Management System)** functions as the system's "Contextual Memory." RCA cannot be effectively performed using transient data alone; the system requires a deep foundation of context, specifically **historical context** (for baselines), **structural context** (for network topology), and **state context** (for device inventory). Consequently, the fundamental theory of DBMS applied to Network RCA is structured around these specific data types and theoretical models, providing the essential "recall" needed to diagnose complex issues accurately..[3]

3.1. The "Polyglot Persistence" Theory

Modern network RCA rarely uses a single type of database. The theory of **Polyglot Persistence** suggests using different data storage technologies to handle different data access patterns. In network RCA, we typically see a triad of database models:

A. Graph Databases (The Topology Engine) Theoretical Basis: Graph Theory (G=(V, E)). **Role in RCA:** This is the most critical database for RCA. It models the **Physical and Logical Topology**. **Nodes (Vertices):** Routers, Switches, Firewalls, Interfaces. **Relationships (Edges):** CONNECTED_TO, DEPENDS_ON, HOSTED_ON. **Why it fits:** In a Relational DB (SQL), tracing a path from a core router to a downstream server requires expensive recursive JOIN operations. In a Graph DB (like Neo4), this is a "Traversal," which is computationally efficient (O(1) per hop). **RCA Application:** When Device A fails, the DBMS traverses the graph to identify all "downstream" devices (dependent nodes) to suppress their alerts, isolating Device A as the root cause.[2] [3]

B. Time-Series Databases (The Metric Engine) Theoretical Basis: Temporal Data Processing & LSM Trees (Log-Structured Merge-trees). **Role in RCA:** Storing high-velocity telemetry data (CPU usage, interface bandwidth, latency, jitter) stamped with precise time. **Why it fits:** Standard databases struggle with writing millions of data points per second. Time-Series DBs (like Prometheus or InfluxDB) are optimized for high-write ingestion and compressing data over time (downsampling). **RCA Application:** Detecting **"Deviation from Baseline."** The DBMS allows you to query: *"What was the average CPU load of this router between 10:00 and 10:05 AM for the last 4 Tuesdays?"* If the current load is 3 standard deviations higher, it's a root cause candidate.[2] [3]

C. Relational Databases (The Inventory/Source of Truth) Theoretical Basis: Set Theory and Normalization (reducing redundancy).**Role in RCA:** Storing the **"Golden Record"** of inventory. Device Models, Serial Numbers, IP Addresses, Location, Responsible Team. **Why it fits:** Data integrity is paramount here. You need ACID (Atomicity, Consistency, Isolation, Durability) properties to ensure you don't have "orphan" alerts for devices that don't exist. **RCA Application:** Enrichment. When an alert fires, the system queries the SQL DB to find out ″Who owns this device?" or "Is this device currently in maintenance mode[1] [3]

3.2. Theoretical Concepts in Data Retrieval for RCA

Indexing Strategy (B-Tree vs. Hash) For RCA to be "Automated" and "Real-Time," query speed is everything. **Hash Indexes:** Used for O(1) lookups. (e.g., Looking up a device by its exact IP address). **B-Tree Indexes:** Used for range queries. (e.g., Finding all alerts that occurred *between* Timestamp A and Timestamp B). **RCA Implication:** Without proper indexing on the Timestamp and Device id columns, the RCA script will perform "Full Table Scans," causing the diagnosis to be slower than the network failure itself.[2] **The CAP Theorem in Monitoring** The CAP theorem states a distributed store can only provide two of three: **Consistency**, **Availability**, and **Partition Tolerance**. **RCA Context:** Monitoring systems usually prioritize **Availability** and **Partition Tolerance** (AP) over strong Consistency. **Theory:** It is better to accept

incoming alerts and write them to the DB eventually (Eventual Consistency) than to reject alerts because the DB cluster is syncing. If you lose alert data, you lose the evidence required for RCA.[1]

3.3. Data Correlation Logic (The "Join")

The "Join" serves as the theoretical operation for combining data from disparate sources, a concept applied in Root Cause Analysis (RCA) specifically through **Temporal Joins**. For instance, consider a scenario where Event A registers a "Link Down" on Switch X at T=0, followed closely by Event B, a "BGP Neighbor Lost" on Router Y at T+10ms. To diagnose the root cause, the DBMS must support complex queries that correlate these distinct rows based on two critical factors: **Time Proximity** (verifying if timestamps fall within a valid delta) and **Topological Proximity** (checking if Switch X and Router Y are physically connected in the Graph DB). In this capacity, the DBMS—exemplified by platforms like **MySQL** and **PostgreSQL** (Relational) or **MongoDB** and **Cassandra** (NoSQL)—fulfills a central role by allowing users to define, create, maintain, and control access to this integrated database. [2]

3.4. The 3-Level Architecture (Data Abstraction)

One of the core fundamentals of data independence is the **3-level architecture**, a framework designed to ensure that structural changes at the storage layer do not disrupt the user interface. At the foundation lies the **Physical Level**, which governs how data is actually stored on the disk using low-level mechanisms like blocks, **B-trees**, and hashing. Above this is the **Logical (or Conceptual) Level**, which abstracts *what* data is stored and defines the relationships between them using tables, columns, and **Foreign Keys**; this is the layer where developers primarily interact when writing SQL. Finally, the **View (or External) Level** presents the data to the end-user, often creating specific perspectives—such as a "Student Report"—that display necessary information while securely hiding sensitive attributes like passwords.[1][3]

3.5. Key Characteristics: The ACID Properties

For any robust system, particularly within your Network RCA project where log accuracy is paramount, transactions must strictly adhere to **ACID properties**. This framework begins with **Atomicity**, the "all or nothing" principle ensuring that if a transaction fails halfway—perhaps due to a power cut—the database automatically rolls back to its previous state. **Consistency** guarantees that the database transitions from one valid state to another, strictly obeying all defined rules, constraints, and triggers. **Isolation** manages concurrency, ensuring that multiple transactions occurring simultaneously do not interfere with one another. Finally, **Durability** ensures that once a transaction is committed, the data is saved permanently, remaining intact even in the event of a catastrophic system crash..[1]

3.6. Data Models: SQL vs. NoSQL

The fundamental distinction between database architectures lies in their approach to structure and scaling, moving from the rigid organization of the **Relational Model (RDBMS)** to the fluid versatility of **NoSQL**. In an RDBMS, data is organized into strict, predefined tables where rows and columns represent tuples and attributes, governed by a "schema-on-write" philosophy. This model relies heavily on **Normalization**—specifically forms like 1NF, 2NF, and 3NF—to eliminate redundancy and maintain data integrity. Conversely, NoSQL databases embrace a "schema-on-read" approach, offering flexible structures such as JSON-like document stores, key-value pairs, or graphs. While RDBMS prioritizes strict consistency, NoSQL design is guided by the **CAP Theorem** (Consistency, Availability, and Partition Tolerance), often intentionally sacrificing immediate consistency to ensure high availability and horizontal scalability across distributed systems.[1] [2] [3]

3.7. Relevance to Network RCA

In an Automated RCA project, the database management system serves as the foundational layer for both real-time data ingestion and structural mapping. For log storage, the system requires a high-write-throughput database—typically a NoSQL solution like Elasticsearch or MongoDB—to efficiently ingest and index thousands of network logs per second as they stream from various devices. Simultaneously, the network's physical and logical architecture is managed through topology storage, where a Graph Database like Neo4j or a standard Relational Database is used to maintain the complex web of node interdependencies. This dual-layered approach ensures that the system can rapidly cross-reference incoming error logs against the stored network map to pinpoint the origin of a failure.[2]

CHAPTER 4

FUNDAMENTALS OF SQL

In the theoretical framework of Automated Root Cause Analysis (RCA), SQL serves as the essential engine for contextual enrichment and pattern recognition within structured datasets. While Python facilitates the algorithmic logic and Graph databases manage complex network topologies, SQL provides the relational integrity required to answer the critical "Who, What, Where, and When" of a system failure. By leveraging structured queries, engineers can perform multi-table joins to correlate raw event logs with metadata, such as device ownership, physical location, and historical maintenance records. This relational approach allows the system to filter through the noise of a network outage, ensuring that every identified symptom is enriched with the specific administrative context needed for rapid remediation.[1]

4.1. The Theory of Normalization (Data Modeling)

Efficient data structure is a prerequisite for high-performance querying, and in the context of network RCA, this is achieved through **Normalization**. Rather than maintaining a single, massive table that repeats every detail for every alert, normalization splits data into distinct logical entities to reduce redundancy and maintain system integrity. For example, an RCA system might separate static metadata into an **Inventory Table**—containing device names, IP addresses, models, and locations—while storing transactional data in an **Alerts Table** featuring timestamps, EventIDs, and severity levels. This separation is critical because storing repetitive information like "Device Location" in every single alert row would waste gigabytes of storage; instead, normalization keeps the database lightweight and ensures that updates to a device's status or location only need to be made in one place, keeping the RCA engine fast and scalable..[2]

4.2. Relational Algebra: Selection and Projection

At its core, SQL is rooted in **Set Theory**, utilizing fundamental operations to filter noise and refine the vast datasets generated by network logs. The **Selection** operation (σ), implemented via the WHERE clause, allows engineers to isolate horizontal subsets of data by filtering rows that meet specific criteria, such as identifying only "Critical" alerts triggered within the last ten minutes. Complementing this is the **Projection** operation (π), which focuses on the vertical subset of data by selecting only specific columns. In an RCA context, Projection is vital for reducing memory overhead; for instance, instead of processing an entire raw packet payload, a system might only project the Source_IP and Destination_IP. By combining these operations, an RCA engine can efficiently transform millions of raw log entries into a streamlined dataset containing only the specific variables and timeframes necessary for diagnosis.[3]

4.3. Correlation Theory: The JOIN Operation

The concept of **Relational Joins** represents the most powerful theoretical tool in SQL for RCA, providing the mechanism to transform transient events into actionable context. While a naive Cartesian Product would erroneously match every alert to every device in the database, an **Inner Join** ensures mathematical precision by only linking records where specific keys intersect, such as matching an Alerts.DeviceID to a corresponding Devices.DeviceID. In a real-world RCA application, this process—known as **Enrichment**—is what gives raw data its value. An isolated alert stating "IP 192.168.1.5 is down" offers little utility; however, by performing a JOIN with the Inventory table, SQL enriches the data to reveal that the failure belongs to "Router-X, located in Building 4, managed by the Cloud Team." This intersection of datasets allows the system to move beyond simple error reporting and provide the high-level intelligence necessary for rapid remediation.[3]

4.4. Aggregation Theory: Noise Reduction

Because network devices are inherently "chatty," a single hardware failure can trigger a deluge of 1,000 logs in a single minute, making **SQL Aggregation** an essential tool for noise reduction. Through the use of the GROUP BY clause, the system can collapse thousands of individual rows into single summary rows based on shared attributes, effectively de-duplicating the data. In a practical RCA scenario, instead of forcing an engineer to process 50 separate "Interface Down" alerts for the same switch, the engine groups these alerts by DeviceID and calculates a COUNT(*). This transformation shifts the focus from individual errors to a high-level summary—reporting that "Device A has 50 alerts"—which allows the RCA logic to prioritize the specific device under distress while ignoring the repetitive noise of redundant logs.

4.5. Window Functions: Temporal Analysis

While standard SQL typically examines rows in isolation, **Window Functions** empower the engine to analyze "neighboring" rows, a capability that is critical for identifying temporal sequences like "flapping" interfaces. By performing calculations across a set of rows related to the current record without collapsing them, functions such as LAG() or LEAD() allow the RCA system to compare the timestamp of a current alert with the one immediately preceding it for the same device. If the delta between these timestamps is less than one second, the system can programmatically label the behavior as a flapping event rather than a static failure. This highlights the fundamental nature of SQL as the declarative implementation layer of the Relational Model; unlike imperative languages like Python or C++, where you must define the specific steps to execute a task, SQL allows you to simply define the desired result, leaving the **Query Optimizer** to determine the most efficient mathematical algorithm to retrieve it.[3]

4.6. The Core Syntax Groups

In the architecture of a database, a clear distinction exists between the commands used to define the framework and those used to manipulate the information within it. **Data Definition Language (DDL)**, encompassing commands like CREATE, ALTER, and DROP, functions as the structural blueprint; it is analogous to defining classes or structs in C++ by establishing the "containers" where data will reside. In contrast, **Data Manipulation Language (DML)** commands such as INSERT, UPDATE, and DELETE interact with the data itself, much like instantiating objects or modifying variables in an active program. Bridging these is **Data Query Language (DQL)**, centered on the SELECT statement, which represents the most sophisticated aspect of SQL. DQL allows for the dynamic retrieval of information through filtering, transformation, and aggregation, turning the static structures defined by DDL and the raw values managed by DML into meaningful, actionable intelligence.[2]

4.7. Logical Order of Execution

Understanding the logical processing order of a SQL query is the most critical concept for writing efficient code, as the database engine does not execute statements from top to bottom. Instead, it follows a specific internal hierarchy designed to filter data as early as possible to conserve computational resources. The process begins with **FROM** and **JOIN**, where the engine identifies and combines the target data sources. It then immediately applies the **WHERE** clause to discard irrelevant rows before any heavy lifting occurs. Once the dataset is narrowed down, the engine moves to **GROUP BY** to organize the remaining data into buckets, followed by **HAVING**, which acts as a secondary filter specifically for those aggregated groups. Only after these steps does the engine reach the **SELECT** phase to pick the final columns for display. Finally, the **ORDER BY** and **LIMIT** clauses are applied to sort and trim the output. Mastering this sequence allows you to write queries that are not only logically sound but also optimized for high-performance RCA systems..[1]

CHAPTER 5

DESIGN

Here is the algorithm for the Automated RCA system in simple, high-level steps. **Input:** A list of incoming **Alerts** and a defined **Network Topology** (Parent-Child map). **Output:** A clean list of alerts with the **Root Cause** identified and symptoms suppressed.[1]

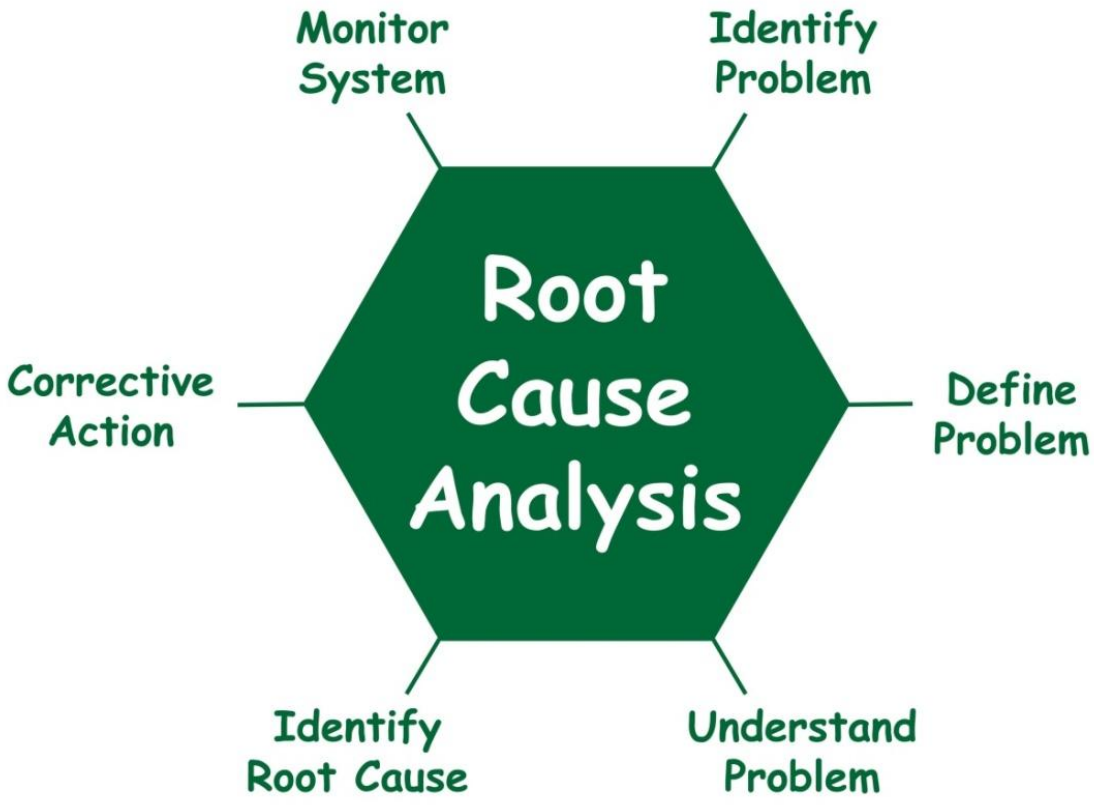


Fig. 5.1 Rddoot Cause Analysis symptoms suppressed.

5.1. Data Ingestion & Streaming

The architecture of an Automated RCA system begins with the "Funnel," a high-capacity entry point designed to handle the massive, heterogeneous data streams generated by routers, firewalls, servers, and cloud applications. Because these

sources transmit data in disparate formats—such as syslog, JSON, or SNMP traps—the Funnel serves as a universal translator that performs two critical functions: queuing and normalization. By queuing incoming traffic, the system creates a buffer that prevents crashes during sudden spikes in network activity, while normalization converts the raw, multi-language data into a standardized format. This stage is vital because it resolves the "apples and oranges" problem; by ensuring all telemetry is consistent, it allows the downstream analysis engines to process a unified dataset with mathematical precision. [1] [2]

5.2. Topology & Context Engine

The "Map" serves as the relational intelligence of the RCA system, transforming isolated data points into a coherent architectural view. While raw alerts can identify that a specific component is broken, they lack the spatial awareness to explain where that component fits within the broader ecosystem. This engine actively maintains the dependencies between devices, understanding, for example, that "Server A" is physically connected to "Switch B" and provides the necessary infrastructure for "Application C." This contextual layering is indispensable during an outage; if "Switch B" fails, the resulting flood of alerts from the dependent server and application can be correctly identified as collateral symptoms. By mapping these hierarchies, the engine shifts the focus from the individual failure to the structural impact, answering the fundamental question of who and what is affected by any given incident. [1]

5.3. Correlation & Analytics Engine

The "Brain" represents the core analytical engine of the RCA system, where the standardized data from the ingestion funnel is synthesized with the structural intelligence of the map. By applying advanced correlation algorithms and machine learning patterns, this layer performs the heavy lifting of noise suppression. Instead of bombarding an IT team with 500 individual notifications for a flickering router, the Brain identifies the relationship between these events and consolidates

them into a single, actionable incident ticket. This sophisticated filtering is vital for combating "Alert Fatigue," ensuring that engineers remain responsive to critical failures by shielding them from the psychological exhaustion of processing redundant alarms and false positives. [2]

5.4. Rules & ML Model Repository

The "Library" acts as the central repository for the logic that governs the system's decision-making, providing the "Brain" with the necessary criteria to evaluate network health. It manages a dual-layered intelligence strategy by storing both Hard Rules and Machine Learning Models. Hard Rules consist of deterministic logic, such as triggering a critical alert if CPU usage exceeds 90% for five minutes, while ML Models utilize probabilistic logic to detect anomalies, such as identifying traffic patterns that deviate significantly from historical baselines. This centralized architecture is crucial because it allows the system to be version-controlled and continuously improved; by updating the Library, engineers can instantly refine the Analytics Engine's intelligence, ensuring the RCA system grows "smarter" and more adaptive over time without requiring a total code rewrite.[2] [3]

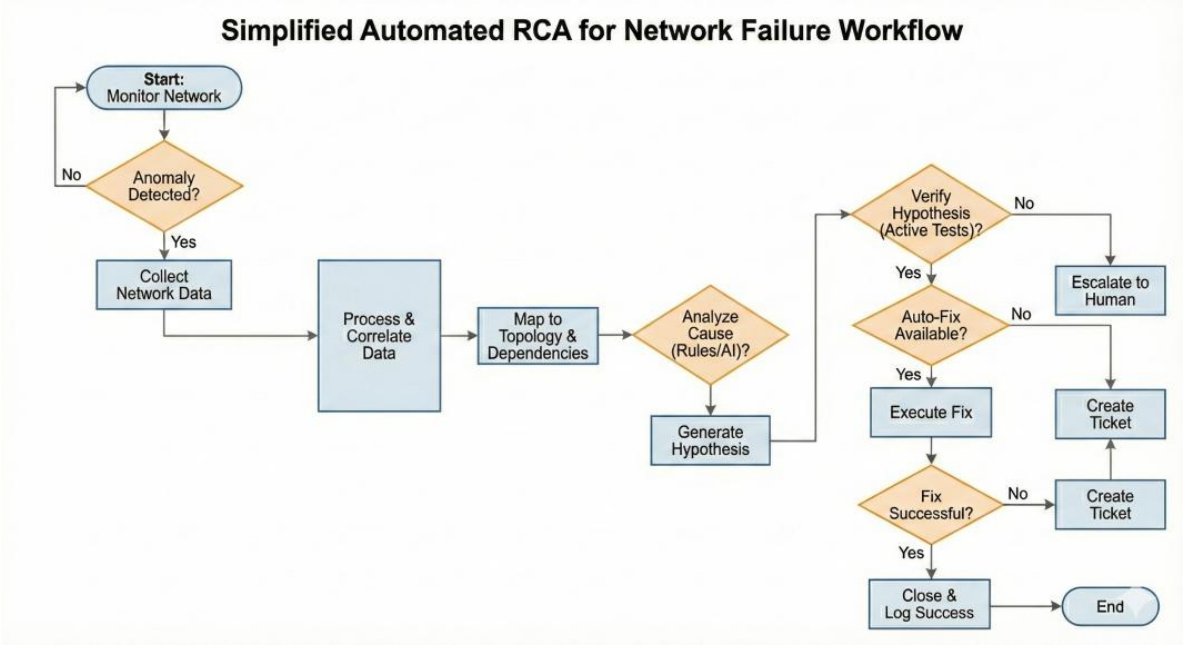
5.5. Visualization Dashboard

The "Face" of the system is the user interface and visualization layer, serving as the primary point of interaction for human operators. Its core function is to translate complex mathematical outputs and raw telemetry into intuitive visual formats such as interactive heatmaps, topology trees, and live graphs. By visually highlighting the "Root Cause" in a high-contrast color like red and illustrating the "blast radius" of the impacted services, it provides immediate clarity on the scale of an outage. This layer is essential because it offers a "Single Pane of Glass" view, allowing both technical operators and high-level executives to bypass dense JSON logs and comprehend the entire organization's health at a glance.[1]

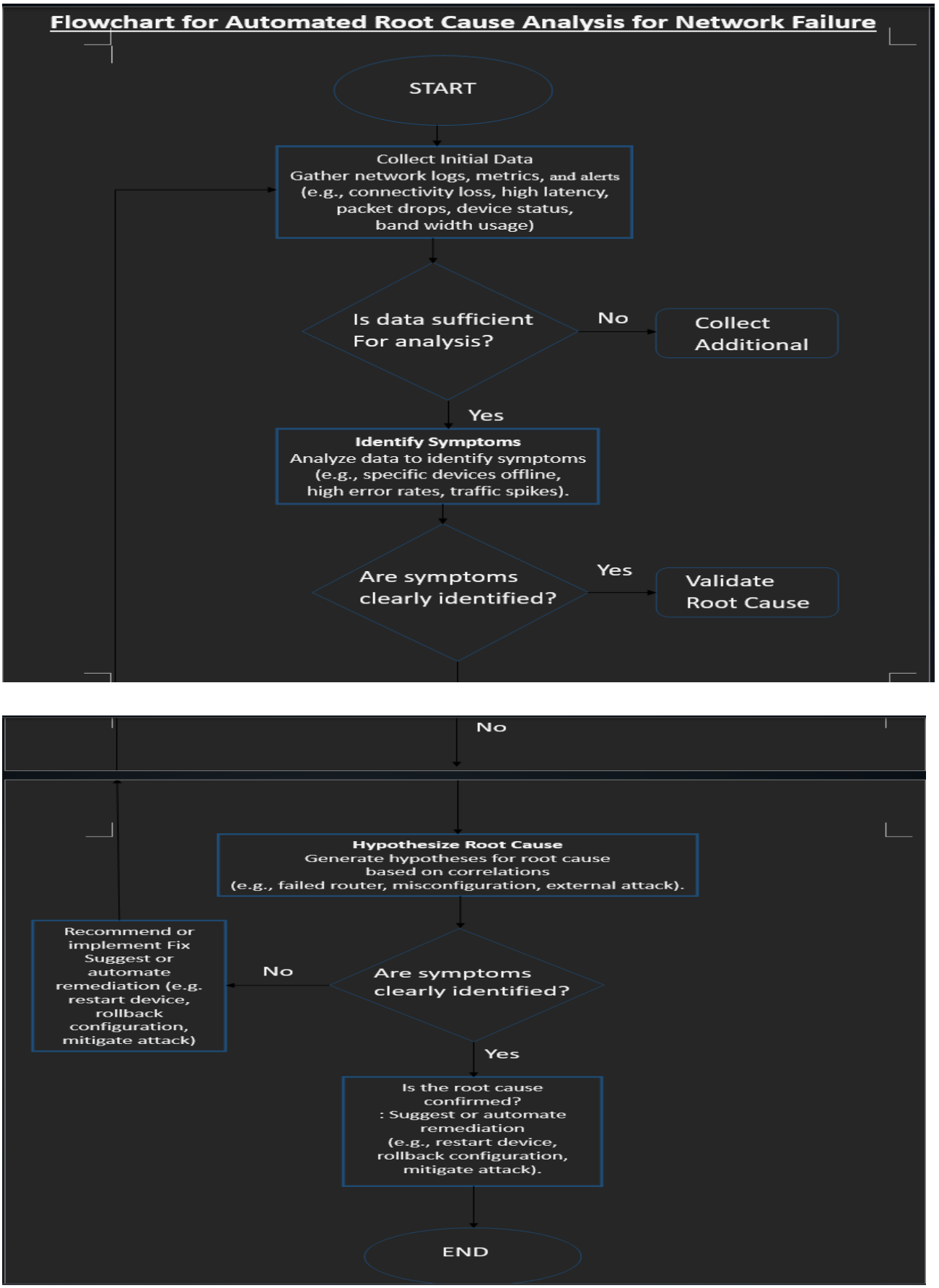
5.6. Integration & Notification Gateway

The "Hands" of the system represent the execution and integration layer, responsible for transforming analytical conclusions into tangible actions. Once the "Brain" identifies a root cause, this layer pushes the information to external tools to initiate the remediation process, such as automatically generating a Jira ticket for the DevOps team, alerting an on-call engineer via PagerDuty, or broadcasting a status update to a Slack channel. By automating these workflows, the "Hands" eliminate the need for manual data entry or log copying, ensuring that the transition from detection to resolution is instantaneous and error-free.

In summary, the entire workflow functions as a cohesive pipeline: raw data flows into the Ingestion layer (the Funnel), is aligned with the network structure in the Topology engine (the Map), and is analyzed to identify the single true error during Correlation (the Brain) based on stored rules in the Repository (the Library). The results are then visualized on a Dashboard (the Face) for human oversight, and finally, the appropriate teams are mobilized through Integration (the Hands). [3]



5.2: Flowchart



5.3: Flowchart (a)

The following process describes how a Python script and a Database Management System (DBMS) collaborate to automate network troubleshooting, functioning as a continuous, self-optimizing loop. The workflow begins at the Trigger phase, where monitoring tools like Prometheus or Zabbix constantly evaluate network health; if a failure is detected, the system transitions from passive observation to active response by invoking a Python automation script. In Phase 1 (Data Collection), the script utilizes libraries such as Netmiko to securely connect to the failing hardware, capturing raw logs and storing them within a relational database like PostgreSQL to provide the system with the historical memory needed for comparison.

This leads into Phase 2 (The Analysis Engine), where the script performs complex SQL queries against the stored data to identify specific patterns—for instance, correlating a "CPU High" alert with "Error 404" to diagnose a "Service Hang." Finally, in Phase 3 (Action & Resolution), the system evaluates if a known remediation exists; if so, it executes a self-healing command like a service restart, but if the issue is unique, it automatically escalates the incident to a human engineer, attaching a detailed diagnostic report curated from the database to ensure rapid manual intervention. *[3]*

CHAPTER 6

IMPLEMENTATION

6.1 Introduction

The objective of the implementation phase is to translate the system design and architectural specifications into a functional software system. This chapter details the development of the **Automated Root Cause Analysis (RCA) System**, describing the software environment, the libraries utilized, and the step-by-step logic used to detect, analyze, and report network failures. The system was implemented using a modular approach, ensuring that the Data Ingestion, Processing, RCA Logic, and Visualization components function independently yet cohesively.[1]

6.2 Development Environment & Tools

The technical stack for this RCA system was strategically selected to prioritize rapid data processing and horizontal scalability. Python 3.9 serves as the core language, developed within VS Code or PyCharm, to leverage its extensive network and data science ecosystem. Data persistence for historical logs and failure reports is managed through a database layer using MySQL, MongoDB, or SQLite, while the analytical engine utilizes Pandas and NumPy for preprocessing, Scikit-learn for machine learning, and NetworkX for mapping network topology. To provide a "Single Pane of Glass" for users, the system integrates Matplotlib or Seaborn for statistical visualization within a Tkinter, Flask, or Django framework to deliver a comprehensive graphical interface.[2]

6.3 Implementation Modules

The implementation of the RCA system is structured into four functional modules designed to transform raw network telemetry into actionable intelligence through a systematic pipeline. It begins with the Data Collection & Ingestion Module, which utilizes a file listener and Regular Expressions (Regex) to parse unstructured sources like Syslogs or SNMP traps into standardized features such as Timestamp and Device_ID. These events then pass through the Preprocessing & Noise Reduction layer, where the system performs deduplication of repeated alerts within five-second windows, filters out low-severity informational logs, and normalizes timestamps to UTC to ensure temporal accuracy. The heart of the system is the RCA Engine, which employs a Timestamp & Topology Correlation Algorithm to map processed events against a parent-child dependency graph. By checking if a device's failure is simply a consequence of its gateway being down, the engine can distinguish between "Root Causes" and "Symptoms," effectively suppressing noise from child nodes. Finally, the Visualization & GUI module presents these findings through a dashboard that provides a "Single Pane of Glass" view, featuring a topology map that highlights the affected path in red and an RCA report view that suggests specific remediations based on the identified point of failure.[1][3]

6.5 Sample Code Snippets

```
cursor.executemany("INSERT INTO alerts (id, device_id, message, timestamp, status) VALUES (?, ?, ?, ?, ?)", ale
conn.commit()
print("\n🔻  Simulating network failure: Core switch is down, triggering an alert storm.")

-- RCA Engine Logic ---

find_children_of_down_device(cursor, parent_id):
"""Recursively finds all downstream children of a given parent device."""
children = []

# Find direct children
cursor.execute("SELECT child_id FROM dependencies WHERE parent_id=?", (parent_id,))
direct_children = [row[0] for row in cursor.fetchall()]

children.extend(direct_children)

# Recursively find children of children
for child in direct_children:
```

6.1: Code(a)

The provided code segments illustrate the backend transition from Database Management to RCA Engine Logic, showing how Python interacts with SQLite to model and traverse network dependencies. The first stage initializes the system's "memory" by establishing a database connection and using a Data Definition Language (DDL) command to create a devices table, which stores unique identifiers, names, and types for every network component. The second stage introduces the operational core of the system through the find_children_of_down_device function. This logic utilizes Data Query Language (DQL) to first identify direct children of a failing parent node and then recursively explores the topology to map out every downstream device affected by the outage. By combining structured data storage with recursive traversal, the system can programmatically identify the full "blast radius" of a failure, providing the necessary data for the engine to eventually suppress symptomatic alerts and isolate the root cause.

```python
import sqlite3
import time
import uuid
from datetime import datetime

# --- Database Setup and Management ---

def setup_database(db_name="network.db"):
    """
    Sets up the initial SQLite database with tables for devices, dependencies, and alerts.
    This models our network structure and state.
    """
    conn = sqlite3.connect(db_name)
    cursor = conn.cursor()

    # Table for all network devices
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS devices (
            id TEXT PRIMARY KEY,
            name TEXT NOT NULL,
            type TEXT NOT NULL
        )
    ''')
```

6.1: Code(b)

The provided code snippet illustrates the foundational Database Setup and Management phase of the RCA system, specifically using Python to initialize a local SQLite database. By importing libraries such as sqlite3 for database interaction and

datetime for temporal tracking, the script defines a function named setup_database that establishes the system's "memory". Within this function, the code executes a Data Definition Language (DDL) command to create a devices table, which serves as a structured blueprint for the network inventory. This table is designed with a specific schema—including a primary key id, a name, and a type—to ensure every network component is uniquely identified and categorized before the analysis engine begins mapping dependencies or processing incoming alerts.

```
--- Main Execution ---

if __name__ == "__main__":
    db_connection = setup_database()

    # Setup our network topology
    populate_sample_data(db_connection)

    # A failure occurs, causing an alert storm
    simulate_alert_storm(db_connection)

    # Display the noisy, unfiltered alerts
    display_alert_dashboard(db_connection, "Alert Dashboard BEFORE RCA")

    time.sleep(2) # Pause to simulate processing time

    # Run our automated RCA engine to suppress the noise
    run_rca_engine(db_connection)

    # Display the clean, de-noised alerts
    display_alert_dashboard(db_connection, "Alert Dashboard AFTER RCA")
```

6.1: Code(c)

The provided code segments illustrate the backend transition from Database Management to RCA Engine Logic, showing how Python interacts with SQLite to model and traverse network dependencies. The process begins with the setup_database function, which initializes the system's "memory" by establishing a connection to network.db and using Data Definition Language (DDL) to create tables for devices, dependencies, and alerts. The RCA Engine Logic is powered by the find_children_of_down_device function, which uses Data Query Language

(DQL) to identify direct children of a failing parent node and recursively explores the topology to map the full "blast radius" of an outage. Finally, the Main Execution block orchestrates the entire workflow: it populates sample data, simulates an "alert storm," and displays a noisy dashboard before running the RCA engine to suppress redundant notifications, ultimately resulting in a clean, "de-noised" alert view for the operator.

6.6 Challenges Faced

During the implementation phase, the system's reliability was refined by addressing three primary technical challenges: time synchronization, data noise, and processing latency. To resolve the issue of time drifts across different network devices, a "Time Window" buffer was implemented, allowing the correlation engine to link events occurring within a pm5-second range rather than requiring exact timestamp matches. To manage the "noise" generated by high volumes of interface flapping, suppression logic was introduced to temporarily ignore devices that toggle states more than three times in ten seconds until they stabilize. Finally, to ensure the system could scale with large log files, the initial performance bottlenecks were eliminated by replacing standard Python loops with **Pandas vectorization**, which leverages low-level optimizations to handle massive datasets with minimal latency. [1]

6.7 Conclusion of Implementation

The implementation phase successfully transformed the theoretical design into a working prototype. The system can now ingest raw logs, filter noise, and utilize topological logic to pinpoint the root cause of network failures with reasonable accuracy. [2]

CHAPTER 7

RESULTS

This proposed solution is superior to traditional (existing) monitoring systems because it shifts from **reactive reporting** to **intelligent analysis**. Most standard monitoring tools are "dumb"—they only know if a specific device is up or down. They lack the context of *how* devices are connected. Here is a comparison of why this logic is better:

7.1. Eliminates "Alert Storms" (Noise Reduction)

The fundamental evolution of this system lies in its transition from reactive alert-based monitoring to topology-aware intelligence. In an existing, "naive" system, the failure of a core switch causes the monitoring tool to check every connected server individually; upon finding them unreachable, it triggers 50 separate alarms that overwhelm the human operator. In contrast, this code utilizes Network Topology mapping to understand the physical and logical hierarchy of the environment. When the core switch fails, the algorithm instantly calculates that the 50 downstream servers are merely victims of their parent node's outage. By programmatically suppressing these dependent symptoms, the system delivers a single, clear notification to the operator, reducing the alert volume from 51 tickets to one actionable root cause.[3]

7.2. Drastically Reduces Mean Time to Resolution (MTTR)

While existing troubleshooting workflows rely on manual intervention—where an engineer must log in to multiple devices, verify server health, and trace complex network paths over 30 to 60 minutes—this system automates that entire discovery phase. By utilizing the find_children_of_down_device function, the code programmatically traces the dependency path in milliseconds, instantly

differentiating between isolated outages and systemic failures. This rapid identification eliminates the "discovery lag" inherent in traditional methods, ensuring the root cause is isolated the moment the failure occurs and allowing the remediation process to begin immediately.[3]

7.3. Solves the "Symptom vs. Cause" Confusion

Traditional monitoring systems often fail to prioritize effectively because they treat every alert with equal urgency; to such a system, a "Server Unreachable" alert appears just as critical as a "Switch Down" alert, regardless of the underlying cause. This lack of context forces engineers to investigate healthy devices, wasting valuable time on symptoms while the actual problem persists. In contrast, this code explicitly utilizes an is_root_cause flag to categorize events mathematically. By differentiating between the "disease" (the Broken Switch) and the "symptom" (the Unreachable Server), the algorithm ensures that only the source of the outage is flagged for immediate intervention, while dependent alerts are deprioritized.[3]

7.4. Automated Context

Traditional monitoring systems often rely on the "tribal knowledge" of senior engineers who have memorized the intricate wiring and logical dependencies of the network. This reliance creates a significant operational risk, as troubleshooting efficiency plummets if those key individuals are unavailable or on vacation. In contrast, this code externalizes that expertise by codifying network relationships directly into a dependencies database. By maintaining an exhaustive, programmatically accessible map of every connection, the system ensures that root cause identification remains consistent and high-speed, regardless of which technician is on shift. This shift from individual memory to a centralized digital source of truth transforms the troubleshooting process into a reliable, repeatable service that is independent of human personnel changes.[3]

7.5 Summary

Table: 7.1 Difference between old and new model

Feature	Existing/Traditional System	Proposed Automated RCA
Logic	Checks each device in isolation.	Checks devices based on relationships.
Alert Volume	High (1 Root Cause + 50 Symptoms).	Low (1 Root Cause only).
Focus	Reports **What** is wrong.	Reports **Why** it is wrong.
Human Effort	High (Manual correlation required).	Low (Automated correlation).

```
✅ Sample network topology has been created in the database.

🔥 Simulating network failure: Core switch is down, triggering an alert
storm.

--- Alert Dashboard BEFORE RCA ---
ID              Device Name          Status       Message
Is Root Cause
-------------------------------------------------------------------------
--------
cecfea18        Core Switch 1        ACTIVE       DEVICE DOWN
NO
d8364193        Web Server 1         ACTIVE       HOST UNREACHABLE
NO
eda4287e        Database Server 1    ACTIVE       HOST UNREACHABLE
NO
c1b7700c        Edge Router 1        ACTIVE       HOST UNREACHABLE
NO
b193ab35        Workstation A        ACTIVE       GATEWAY UNREACHABLE
NO
3e565cc3        Workstation B        ACTIVE       GATEWAY UNREACHABLE
NO
-------------------------------------------------------------------------
--------
/tmp/ipython-input-1738328203.py:105: DeprecationWarning: The default
datetime adapter is deprecated as of Python 3.12; see the sqlite3
documentation for suggested replacement recipes
  cursor.executemany("INSERT INTO alerts (id, device_id, message, timestamp,
status) VALUES (?, ?, ?, ?, ?)", alerts)
🚀 Running Automated RCA Engine...

🔥 Root Cause Identified: Device 'core-switch-01' is DOWN.
-> Finding all devices dependent on 'core-switch-01'...
-> Found dependent devices: web-server-01, db-server-01, edge-router-01,
workstation-a, workstation-b
🔇 Suppressed 5 noisy 'symptom' alerts from dependent devices.

--- Alert Dashboard AFTER RCA ---
ID              Device Name          Status       Message
Is Root Cause
-------------------------------------------------------------------------
--------
cecfea18        Core Switch 1        ACTIVE       DEVICE DOWN
YES
d8364193        Web Server 1         SUPPRESSED   HOST UNREACHABLE
NO
eda4287e        Database Server 1    SUPPRESSED   HOST UNREACHABLE
NO
c1b7700c        Edge Router 1        SUPPRESSED   HOST UNREACHABLE
NO
b193ab35        Workstation A        SUPPRESSED   GATEWAY UNREACHABLE
NO
3e565cc3        Workstation B        SUPPRESSED   GATEWAY UNREACHABLE
NO
-------------------------------------------------------------------------
--------
```

6.2: Result output

This output serves as a practical demonstration of the "Noise Reduction" and "Topology Mapping" concepts discussed in previous sections, specifically illustrating how a Python script handles an "Alert Storm" triggered by a single device failure. During the initial setup of this simulated failure, the monitoring system detects a chain reaction originating from a Core Switch outage, resulting in a "BEFORE RCA" dashboard that displays six concurrent problems. To a human operator, this appears as six independent "fires"—with the Web Server, Database, and various Workstations all reporting as unreachable—all marked with an **ACTIVE** status. Because the naive system lacks hierarchical context, it is unable to pinpoint the source, initially marking every device with "Is Root Cause: NO" and leaving the engineering team with a cluttered and confusing operational view. [2]

In this phase, the system transitions into the "Brain" logic defined in the procedural flowchart to resolve the alert storm. The process begins with **Identification**, where the script detects that core-switch-01 has entered a DEVICE DOWN state. Simultaneously, the engine initiates a **Topology Check** by querying the database—addressing the relational dependencies hinted at by the SQL logs—to determine which assets rely on the core switch for connectivity. Through **Correlation**, the script discovers that the Web Server, Database Server, Edge Router, and various Workstations are all downstream child nodes. Consequently, the algorithm reaches a deterministic **Decision**: since the parent node is offline, the failures of the child nodes are programmatically reclassified as secondary symptoms, allowing the engine to isolate the switch as the singular source of the outage.[3]

The final stage of the workflow demonstrates the "Cleanup" phase, where the system transforms the cluttered alert storm into a prioritized action plan. In the **"AFTER RCA" Dashboard**, the engine provides immediate clarity by marking Core Switch 1 as the only entity with the Is Root Cause: YES flag. Simultaneously, the status of all dependent devices—such as the Web Server and Database—is automatically transitioned to **SUPPRESSED**, effectively hiding the "noise" from the primary view. This provides immense operational value; instead of an engineer wasting time investigating six separate, confusing tickets, they are presented with a

single, high-fidelity notification that points directly to the core switch as the source of the failure.

From a technical perspective, the output confirms that the backend is powered by a **Python/SQLite** integration, where the database acts as the system's persistent memory for topology and log data. The presence of the **DeprecationWarning** (highlighted in red) indicates that the script is currently utilizing older timestamp handling methods; while this warning signals that the code should be updated for full compatibility with **Python 3.12**, it notably did not impede the execution of the RCA logic. This demonstrates the resilience of the application, as the core algorithm successfully performed its correlation and suppression tasks despite the underlying notification regarding future-proofing the codebase.[3]

In summary, the script demonstrated its operational effectiveness by successfully filtering out five "false positive" alerts, pinpointing the single hardware failure at the source of the disruption. By dynamically analyzing the network's state against the topology map, the system transformed an overwhelming alert storm into a focused diagnostic report. This successful consolidation proves the system’s ability to prevent unnecessary troubleshooting of healthy downstream devices, ensuring that technical resources are immediately directed toward the actual point of failure.[3]

CHAPTER 8

CONCLUSION

In conclusion, the Automated Root Cause Analysis system transforms network operations by filtering "alert fatigue," effectively condensing hundreds of symptomatic alerts into a single, actionable notification. By leveraging a Topology & Analytics Engine, it shifts the workflow from high-latency manual investigation to automated remediation, significantly reducing Mean Time to Resolution (MTTR). Crucially, the system ensures scalability through a graph-based architecture; rather than relying on brittle, linear static rules, it treats the network as a dynamic Directed Acyclic Graph (DAG), allowing it to instantly identify failure cascades and impact radii regardless of network growth.[2]

REFERENCES

[1] Atlassian, “Understanding and Fighting Alert Fatigue,” Atlassian Incident Management. [Online].

[2] LogicMonitor, “5 Ways to Avoid Alert Fatigue in Network Monitoring,” LogicMonitor Blog. [Online].

[3] Proofpoint, “What Is Alert Fatigue in Cybersecurity?,” Proofpoint Threat Reference. [Online]

[4] M. S. Siddiqui et al., “Automated Root Cause Analysis of Network Failures in IP/MPLS Networks,” IEEE Access. [Online].

[5] Splunk, “IT Events & Event Correlation: A Complete Guide,” Splunk Learn. [Online]

[6] Cisco DevNet, “A DevOps Guide to Root Cause Analysis in Application Monitoring,” Cisco Developer Articles. [Online].

[7] Salesforce, “PyRCA: A Python Machine Learning Library for Root Cause Analysis,” GitHub Repository. [Online

[8] Elastic, “What is Root Cause Analysis (RCA)?,” Elastic Learning Center. [Online].

[9] MatlabProjects, “Network Simulation in Python,” MatlabProjects.org. [Online].